GET STARTED TODAY

LUKE GINN

AZURE CLOUD MASTERY

Unit 16 Introduction – Why Agents Need Memory

AZURE CLOUD MASTERY

Unit 15 covered performance evaluation. Unit 16 covers agent memory – storing information across conversation turns and user sessions.

- **Review of Unit 15**: You learned to measure agent quality. Now you need to give agents memory so they remember past conversations and user preferences.
- **What This Unit Covers**: You will learn short-term memory (session context) vs. long-term memory (user preferences across sessions), and persistence layers (Cosmos DB, Redis).
- **Why Memory Matters**: Without memory, agents forget everything after each message. They cannot have natural conversations or remember user preferences.

Exam Takeaway: Agent Memory is part of Observability domain (20% weighting). The exam tests short-term vs. long-term distinction and persistence coding patterns.

Short-Term Memory – Session Context

AZURE CLOUD MASTERY

Short-term memory stores information for the duration of a single conversation session, resetting when the session ends or times out.

- **Short-Term Definition**: The agent remembers what was said 5 messages ago within the same conversation. Information is stored in memory, not re-sent with each prompt.
- **Session Expiration**: A session typically expires after 1 hour of inactivity or when the user explicitly ends the conversation. Memory is then cleared.
- **Storage Location**: Short-term memory lives in the agent's runtime memory (RAM) or in a fast cache like Redis. Not written to long-term database.

Exam Takeaway: Short-term memory = same session only. Expires after inactivity or session end. The exam tests when to use short-term vs. long-term.

Long-Term Memory – User Preferences Across Sessions

AZURE CLOUD MASTERY

Long-term memory stores information that persists across multiple conversation sessions, such as user preferences, history, or profile data.

- **Long-Term Definition**: The agent remembers "User prefers Celsius" or "User's shipping address is 123 Main St" even if the user returns next week.
- **Persistence Layer**: Long-term memory is written to a database (Cosmos DB) or key-value store (Redis with persistence). Data survives agent restarts.
- **User Identification**: Long-term memory requires a stable user ID (from Entra ID or session cookie) to retrieve the correct user's data on each conversation.

Exam Takeaway: Long-term memory = across sessions. Requires user ID and persistent storage (Cosmos DB, Redis). The exam tests storage selection.

Short-Term vs. Long-Term – When to Use Each

AZURE CLOUD MASTERY

Use short-term memory for conversational context within a session. Use long-term memory for user preferences and history that should persist.

- **Short-Term Examples**: "You asked about refunds earlier, now you are asking about exchanges." The agent connects the topics. Not saved across sessions.
- **Long-Term Examples**: "You prefer dark mode. Your default shipping address is 123 Main St. You have requested refunds on 3 previous orders."
- **Memory Hybrid**: Agent first checks long-term memory for user preferences at session start. During the session, short-term memory tracks the current conversation.

Exam Takeaway: The exam tests identifying whether a piece of information belongs in short-term (session-only) or long-term (cross-session) memory.

Conversation History as Short-Term Memory

AZURE CLOUD MASTERY

The most basic form of short-term memory is storing the conversation history – all previous user and agent messages – and sending them with each LLM call.

- **How It Works**: Agent stores messages in an array: `messages = [{"role": "user", "content": "Hi"}, {"role": "assistant", "content": "Hello"}]`.
- **Appending to Prompt**: Before each LLM call, agent sends the entire `messages` history plus the new user message. The LLM sees the full conversation.
- **Token Limit Caution**: Long conversations exceed token limits. Implement sliding window: keep only last 10 messages or summarize older messages.

Exam Takeaway: Conversation history is short-term memory. Token limits require truncation or summarization. The exam tests token limit management.

Sliding Window – Managing Token Limits

AZURE CLOUD MASTERY

A sliding window keeps only the most recent N messages in short-term memory, discarding older messages when the conversation becomes too long.

- **Sliding Window Definition**: Set maximum messages to 20. When message 21 arrives, discard message 1. Window always contains most recent 20 messages.
- **Token-Based Window**: Instead of message count, track token count. Keep adding messages until token limit (e.g., 8,000 tokens reached), then remove oldest.
- **What Is Lost**: When you discard old messages, the agent loses that context. The user may need to repeat information. Inform user: "Restarting conversation to save memory."

Exam Takeaway: Sliding window prevents token overflow but loses context. The exam tests implementing sliding window logic and handling lost context.

Summarization as Short-Term Memory Compression

AZURE CLOUD MASTERY

Instead of discarding old messages, you can summarize them into a shorter form, preserving key information while reducing token count.

- **Summarization Flow**: After every 10 messages, call an LLM with prompt: "Summarize this conversation so far, keeping key facts and user intent."
- **Replace with Summary**: Replace the original 10 messages with the 200-token summary. Continue appending new messages. Total token count stays within limits.
- **Lossy Compression**: Summaries lose some details. The agent may forget minor points but retains major facts and user preferences.

Exam Takeaway: Summarization preserves context better than discarding, but loses detail. The exam tests choosing between sliding window (simple) vs. summarization (better context).

Azure Cosmos DB for Long-Term Memory

AZURE CLOUD MASTERY

Azure Cosmos DB is a NoSQL database ideal for storing long-term agent memory because it scales infinitely and supports JSON documents.

- **Why Cosmos DB**: Cosmos DB stores JSON directly. Agent memory (user preferences, conversation history) is already JSON. No conversion needed.
- **Partition Key for Users**: Use `user_id` as partition key. All memories for a user are stored together, enabling fast retrieval.
- **Time-to-Live (TTL)**: Set TTL on memory documents. Example: "User preferences expire after 90 days of no interaction." Cosmos DB automatically deletes expired memories.

Exam Takeaway: Cosmos DB is the recommended long-term memory store. Use user_id as partition key. TTL auto-expires old memories.

Coding Pattern – Storing Memory in Cosmos DB

AZURE CLOUD MASTERY

Your agent code uses the Cosmos DB SDK to save memory objects as JSON documents, using user_id as partition key and conversation_id as document ID.

- **SDK Pattern**: `from azure.cosmos import CosmosClient`. `container.upsert_item({"id": conversation_id, "user_id": user_id, "memory": memory_json, "ttl": 7776000})`.
- **Upsert Behavior**: `upsert_item` creates the document if it does not exist, or replaces it if it does. Use for updating memory.
- **Reading Memory**: `existing = container.read_item(item=conversation_id, partition_key=user_id)`. Returns the memory JSON. Handle `CosmosResourceNotFoundError` for new users.

Exam Takeaway: The coding expectation is using `upsert_item()` to save memory and `read_item()` to retrieve it. Handle missing document errors.

Azure Managed Redis for Short-Term Memory

AZURE CLOUD MASTERY

Azure Managed Redis is an in-memory cache ideal for short-term memory because it is extremely fast (sub-millisecond latency) and supports expiration.

- **Why Redis for Short-Term**: Redis stores data in RAM, not disk. Retrieval is much faster than Cosmos DB (1ms vs 50ms). Perfect for session memory.
- **Expiration Built-In**: Set Redis key to expire after 1 hour: `SETEX session:user123 "conversation history" 3600`. Key auto-deletes after 1 hour of inactivity.
- **Redis vs. Cosmos DB**: Use Redis for short-term (session memory, conversation history). Use Cosmos DB for long-term (user preferences, cross-session data).

Exam Takeaway: Redis is for fast, short-term, expiring memory. Cosmos DB is for durable, long-term memory. The exam tests which service for which purpose.

Coding Pattern – Redis for Session Memory

AZURE CLOUD MASTERY

Your agent code uses the Redis SDK to store session memory with a time-to-live (TTL), and retrieves it at the start of each conversation turn.

- **SDK Pattern**: `import redis; r = redis.Redis(host="mycache.redis.cache.windows.net", password=key)`. `r.setex(f"session:{user_id}", 3600, json.dumps(memory))`.
- **Retrieving Memory**: `memory_json = r.get(f"session:{user_id}")`. If `None` returned, no session exists – create new memory object.
- **Updating Memory**: After each agent turn, update the stored memory with new conversation history and call `r.setex()` again to refresh the 1-hour TTL.

Exam Takeaway: Use `setex()` to set key with expiration. Check for `None` to detect new sessions. Refresh TTL on every interaction.

Memory Structure – What to Store

AZURE CLOUD MASTERY

A well-structured memory object contains conversation history, user preferences, session state (e.g., awaiting approval), and any intermediate results.

- **Required Fields**: `conversation_id`, `user_id`, `messages` (list of recent messages), `preferences` (dictionary, e.g., `{"temperature_unit": "C"}`).
- **Session State Fields**: `state` (e.g., "awaiting_order_number", "approval_pending"), `pending_action` (tool call awaiting result), `last_activity_timestamp`.
- **Size Limits**: Keep memory under 10,000 tokens for Cosmos DB (2MB document limit) and under 1MB for Redis to maintain performance.

Exam Takeaway: Memory must include conversation history, preferences, and session state. The exam tests designing memory schemas with required fields.

Serializing and Deserializing Memory JSON

AZURE CLOUD MASTERY

Your agent code must serialize (convert) memory objects to JSON strings before storing, and deserialize (parse) JSON strings back to objects after retrieval.

- **Serialization**: `memory_json = json.dumps({"user_id": "123", "preferences": {"unit": "C"}, "messages": messages_list})`.
- **Deserialization**: `memory = json.loads(memory_json)`. Access fields: `user_id = memory["user_id"]`, `preferences = memory["preferences"]`.
- **Handling Missing Fields**: Use `memory.get("preferences", {})` to provide default empty dict if field does not exist in older stored memories.

Exam Takeaway: The coding expectation is using `json.dumps()` before storage and `json.loads()` after retrieval. Handle missing fields with `.get()`.

Connecting Strings and Endpoints for Memory Services

AZURE CLOUD MASTERY

Both Cosmos DB and Redis require connection strings or endpoints with authentication keys, stored securely in environment variables or Key Vault.

- **Cosmos DB Connection String**: `AccountEndpoint=https://your-account.documents.azure.com;AccountKey=your-secret-key`. Store in `COSMOS_CONNECTION_STRING`.
- **Redis Connection String**: `mycache.redis.cache.windows.net:6380,password=your-key,ssl=True`. Store in `REDIS_CONNECTION_STRING`.
- **Managed Identity for Cosmos DB**: Enable managed identity on Agent Service. Grant "Cosmos DB Built-in Data Contributor" role. No connection string needed.

Exam Takeaway: Store connection strings in environment variables. Prefer managed identity for Cosmos DB. The exam tests secure credential storage.

Foundry Trace for Memory Operations

AZURE CLOUD MASTERY

Foundry Trace records memory read and write operations as spans, showing latency and whether the operation succeeded or failed.

- **Memory Read Span**: Span name `"memory_read"`. Attributes: `store_type` (redis/cosmos), `key`, `hit` (true if data found, false if new session), `duration_ms`.
- **Memory Write Span**: Span name `"memory_write"`. Attributes: `store_type`, `key`, `size_bytes` (size of serialized memory), `ttl_seconds`, `duration_ms`.
- **Debugging Slow Memory**: If memory operations exceed 100ms, trace shows the span. Investigate network latency or large memory size. Reduce memory size or move to faster region.

Exam Takeaway: Instrument memory operations with spans. The exam tests adding attributes for store type, hit/miss, size, and latency.

Unit 16 Summary – Agent Memory

AZURE CLOUD MASTERY

You have learned to implement short-term memory (session context) with Redis and long-term memory (user preferences) with Cosmos DB, using JSON serialization.

- **Short-Term Memory**: Session-only context. Use Redis for speed and TTL expiration. Store conversation history and session state. Expires after inactivity.
- **Long-Term Memory**: Cross-session preferences. Use Cosmos DB for durability. Partition by user_id. Use TTL to auto-expire old memories.
- **Coding Skills**: Serialize memory with `json.dumps()`, deserialize with `json.loads()`. Use `setex()` for Redis expiration. Use `upsert_item()` for Cosmos DB.
- **Observability**: Foundry Trace records memory operation spans with store type, key, size, and latency.
- **Next Unit Preview**: Unit 17 introduces Multimodal Integration – Vision, Speech, and Document Intelligence as agent tools.

Exam Takeaway: Agent Memory is 20% of the exam. Master short-term vs. long-term selection, Redis vs. Cosmos DB, JSON serialization, and trace instrumentation.

AZURE CLOUD MASTERY

THANKS FOR WATCHING